MEMORIA TÉCNICA DEL PROYECTO

Sistema Distribuido de Gestión de Riego Urbano (WaterManagement — SD)

Asignatura: Sistemas Distribuidos
Entorno de Ejecución: Python 3.10+, Docker Desktop, Apache Kafka (KRaft), Railway Cloud
Repositorio: `WaterManagement-SD`

1. Resumen Ejecutivo y Objetivos del Proyecto

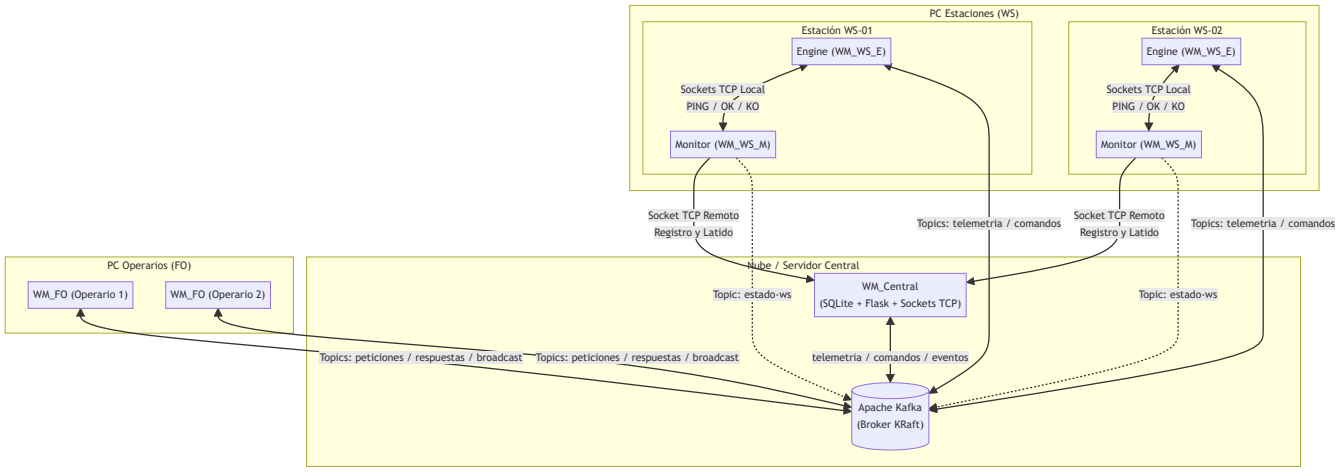
El proyecto **WaterManagement (WM)** es una solución distribuida, tolerante a fallos y orientada a eventos para la monitorización, supervisión y control del riego automatizado en parques y zonas verdes urbanas.

El sistema modela y resuelve de forma desacoplada la interacción entre cuatro entidades fundamentales:
1. **WM_Central (Centro de Control):** Núcleo de supervisión que coordina la red, persiste el estado global en base de datos relacional (SQLite), expone un panel web interactivo para operarios y administradores, y gestiona tanto sockets directos como colas de mensajería asíncrona.
2. **WM_WS_M (Watering Station Monitor):** Proceso supervisor de cada punto de riego. Se autentica ante la Central mediante sockets TCP, supervisa la salud operativa de su actuador local (Engine) mediante un protocolo de latido (*heartbeat*) y notifica incidentes a la Central.
3. **WM_WS_E (Watering Station Engine):** Proceso actuador que simula la electroválvula de riego, computa y reporta en tiempo real telemetría volumétrica y de caudal hacia la Central, responde a comandos remotos (iniciar, parar, bloquear, activar) y posee capacidad de **auto-riego periódico autónomo**.
4. **WM_FO (Field Operator):** Aplicación de campo para operarios de jardinería que permite consultar el estado de la red y emitir órdenes de riego puntuales o por lotes (mediante scripts JSON secuenciales con retardos controlados).

Objetivos Técnicos Alcanzados

- **Comunicación Híbrida Justificada:** Integración de sockets TCP de bajo nivel con tramas delimitadas y detección de errores (LRC) junto a un middleware de mensajería desacoplada basada en eventos (Apache Kafka).
- **Concurrencia Segura:** Arquitectura multihilo en todos los nodos (Locks, Events, hilos Daemon) sin condiciones de carrera ni interbloqueos (*deadlocks*).
- **Tolerancia a Fallos y Alta Disponibilidad:** Mecanismos de reconexión automática ante caídas de red, reinicio consistente de la Central con resincronización de estado, y blindaje de hilos de consumo frente a tramas malformadas.
- **Despliegue Multi-Entorno:** Operatividad garantizada tanto en entorno local mono-máquina, como en red local de 3 ordenadores independientes, y en infraestructura Cloud (Railway con TCP Proxies).

2. Arquitectura del Sistema y Decisiones de Diseño



2.1. Dualidad de Canales: Sockets TCP directos vs. Apache Kafka

Una de las decisiones arquitectónicas centrales es la separación entre el canal síncrono punto a punto (Sockets TCP) y el canal asíncrono desacoplado (Kafka):

1. **¿Por qué Sockets TCP para Monitor ↔ Central y Monitor ↔ Engine?**

2. **Semántica de presencia y latido implícito:** Mantener un socket TCP abierto proporciona una garantía física de conexión. Si el cable se desconecta o el proceso muere, el kernel del sistema operativo o el timeout del socket señalan inmediatamente el corte sin depender de intermediarios.
3. **Handshake estricto de identidad:** El registro de una estación requiere una respuesta síncrona inmediata (`REGISTRO` → `REGISTRO_OK`) antes de que el actuador comience a operar.
4. **Aislamiento local estación:** La comunicación entre el Monitor y el Engine ocurre en la misma máquina o en red local cercana. No tiene sentido saturar el bus global de mensajería con *pings* de salud de 1 Hz entre dos procesos locales.

5. **¿Por qué Apache Kafka para Telemetría, Comandos y Operarios?**

6. **Desacoplamiento Temporal:** Un operario puede enviar una petición de riego o desconectarse tras emitirla; los mensajes de confirmación o resumen se conservan en el log persistente de Kafka y son consumidos en cuanto el operario vuelve a estar disponible (`desde_inicio=True`).
7. **Escalabilidad y Difusión:** La telemetría de decenas de estaciones concurrentes (1 mensaje/segundo por estación regando) y el *broadcast* periódico del estado de la red se distribuyen a múltiples suscriptores sin sobrecargar los sockets de Central.
8. **Persistencia y Repetición:** Si Central se reinicia temporalmente, los mensajes encolados en Kafka no se pierden.

2.2. Protocolo de Tramas de Sockets de Nivel de Aplicación

Dado que TCP es un protocolo orientado a flujo de bytes continuo (*stream*), carece del concepto de delimitador de mensaje. Si dos mensajes se emiten consecutivamente, pueden recibirse agregados o fragmentados. Para solventar este problema clásico, se implementó en `utils/protocol.py` una especificación de trama estandarizada:

```
+-------+-----------------------------+-------+-----+
|  STX  |     PAYLOAD UTF-8           |  ETX  | LRC |
| 0x02  | "REGISTRO#WS-01#River Park" | 0x03  | 1 B |
+-------+-----------------------------+-------+-----+
```

- **STX (`\x02` - Start of Text):** Sincroniza al receptor marcando el inicio del mensaje.
- **Payload:** Texto codificado en UTF-8 con campos delimitados por `#` .
- **ETX (`\x03` - End of Text):** Señaliza el fin del contenido útil.
- **LRC (Longitudinal Redundancy Check):** Checksum de 1 byte calculado mediante XOR acumulativo sobre todos los bytes del payload:
  $$\text{LRC} = b_0 \oplus b_1 \oplus b_2 \oplus \dots \oplus b_{n-1}$$
  Si durante la recepción el LRC recalculado difiere del recibido, la trama se rechaza con un byte de control `NACK` (`\x15`), solicitando retransmisión.
- **Caracteres de control mono-byte:** Se reservan bytes específicos fuera de trama para el diálogo de enlace rápido: `ENQ` (`\x05` - interrogación/latido), `ACK` (`\x06` - confirmación positiva), `NACK` (`\x15` - fallo/reintento) y `EOT` (`\x04` - fin de transmisión).

2.3. Topología de Topics en Apache Kafka

Todos los tópicos se centralizan en `utils/kafka_utils.py` para evitar inconsistencias de nomenclatura:

Nombre del Topic	Origen	Destino	Payload / Semántica	Particionado
`telemetria-riego`	Engine (`WM_WS_E`)	Central (`WM_Central`)	`{"tipo": "TELEMETRIA", "id_ws", "caudal_lmin", "volumen_total_l"}` y `{"tipo": "RIEGO_FINALIZADO", "motivo", ...}`	`key=id_ws`
`comandos-estacion`	Central (`WM_Central`)	Engine (`WM_WS_E`)	`{"comando": "INICIAR_RIEGO"|"DETENER_RIEGO"|"BLOQUEAR"|"ACTIVAR", "id_ws", "duracion_seg", ...}`	`key=id_ws`
`estado-ws`	Monitor (`WM_WS_M`)	Central (`WM_Central`)	`{"evento": "FUGA"|"FUGA_RESUELTA"|"CONECTADO"|"DESCONECTADO", "id_ws"}`	`key=id_ws`
`riego-peticiones`	Operario (`WM_FO`)	Central (`WM_Central`)	`{"tipo": "SOLICITUD_RIEGO", "request_id", "id_operador", "id_ws", "duracion_seg"}`	`key=id_operador`
`riego-respuestas`	Central (`WM_Central`)	Operario (`WM_FO`)	`{"evento": "AUTORIZADO"|"DENEGADO"|"RESUMEN"|"INTERRUMPIDO", "request_id", "id_operador"}`	`key=id_operador`
`central-broadcast`	Central (`WM_Central`)	Operarios / Monitores	`{"estaciones": [...], "timestamp": ...}` emitido cada 2 segundos con la foto global de la red	Aleatorio

Decisiones de Diseño en Kafka:

1. **Garantía de orden por `key` :** Al publicar con `key=id_ws` en `comandos-estacion` , Kafka garantiza que todos los comandos dirigidos a una misma estación van a parar a la misma partición, impidiendo que una orden `INICIAR_RIEGO` adelante a un `BLOQUEAR` .
2. **Políticas de Grupos de Consumo (`group_id`):**
3. **Central:** Emplea grupos estables (`group_id="central-peticiones"` , `group_id="central-telemetria"`). Al reiniciar Central, Kafka recuerda el último *offset* confirmado y no se procesan peticiones duplicadas.

4. **Operarios (FO):** Emplean identificadores de grupo únicos efímeros (`group_id=f"fo-{id_operador}-{uuid4()}"`) configurados con `desde_inicio=True`. Si el operario se incorpora tarde o cuando el riego ya ha finalizado, recupera el historial del topic y captura el mensaje final `RESUMEN`.
5. **Broadcast:** Utiliza `group_id=None` y `auto_offset_reset='latest'`. Como el estado se reemite periódicamente cada 2 segundos, el operario solo consume el estado más fresco.

2.4. Modelo de Persistencia y Concurrencia (SQLite)

La persistencia reside en `utils/db.py` sobre SQLite local (montado sobre volumen persistente en Railway en `/app/data/wm_central.db`):

- **Control de Acceso Multihilo:** SQLite no admite escrituras concurrentes desde hilos arbitrarios sin coordinación. Para resolverlo se utiliza un cerrojo de exclusión mutua global de Python (`threading.Lock()`) instanciado a nivel de módulo:
  `python _lock = threading.Lock() with _lock: self._conn.execute(...) self._conn.commit()`
- **Idempotencia con UPSERT:** Tanto el registro de estaciones como la incorporación de operarios utilizan cláusulas `ON CONFLICT(...) DO UPDATE` / `DO NOTHING`. Esto evita excepciones de clave primaria duplicada ante reconexiones sucesivas de estaciones.
- **Filtro Defensivo en Telemetría:** La actualización de caudal en BD aplica un filtro estricto:
  `sql UPDATE estaciones SET estado='REGANDO', caudal=?, volumen=? WHERE id_ws=? AND estado IN ('DISPONIBLE', 'REGANDO')`
  Si un administrador ha bloqueado la estación desde el panel (`FUERA_DE_SERVICIO`), una trama rezagada de telemetría no "resucita" accidentalmente la estación a `REGANDO`.

2.5. Resiliencia, Auto-Riego y Auto-Recuperación

1. **Reinicio de Central y Borrado de Estados Volátiles:**
   Al arrancar Central, se ejecuta inmediatamente `db.marcar_todas_desconectadas()`. Esta decisión de diseño es crítica: hasta que un Monitor no vuelva a establecer el socket y confirmar la presencia de su Engine, Central no puede inventarse el estado operativo de los equipos. En un plazo de 5 a 15 segundos, los Monitores detectan la reconexión y notifican su estado real (`CONECTADO` o `FUGA`).
2. **Capacidad de Auto-Riego Autónomo en la Estación:**
   El Engine (`WM_WS_E.py`) incorpora un hilo dedicado `hilo_auto_riego`. Mediante las variables de entorno `AUTO_RIEGO_INTERVALO` y `AUTO_RIEGO_DURACION`, la estación puede auto-iniciar riegos sin requerir intervención de operarios ni órdenes directas de Central. Durante el auto-riego, la telemetría se emite registrando a `"ESTACION"` como operario ejecutor. Si la estación sufre una avería (`FUGA`) o es bloqueada administrativamente, el auto-riego se pospone automáticamente.
3. **Control Administrativo Centralizado:**
   Incluso con operarios ausentes o dados de baja en la tabla `operarios`, la Central dispone de botones de mando en el panel web (`Regar`, `Bloquear`, `Activar`). El riego manual emitido por Central se registra asociando al operador especial `"CENTRAL"`.

---

3. Explicación Detallada del Código Fuente

A continuación se documenta la responsabilidad, estructura y mecanismos internos de cada fichero del repositorio:

```
WaterManagement-SD/
├── utils/
│   ├── protocol.py           # Protocolo STX/ETX/LRC y sockets de bajo nivel
│   ├── kafka_utils.py        # Clientes y consumidores Kafka con reintentos
│   └── db.py                 # Capa DAL SQLite thread-safe
├── WM_Central/
│   ├── WM_Central.py         # Orquestador central multihilo
│   ├── dashboard.py          # Backend web Flask (REST + polling)
│   ├── templates/
│   │   └── dashboard.html    # Frontend HTML5 + Bootstrap + Polling JS
│   └── Dockerfile            # Imagen contenedor de Central
├── WM_WS/
│   ├── WM_WS_M.py            # Monitor de estación (supervisor)
│   ├── WM_WS_E.py            # Engine de estación (actuador + auto-riego)
│   ├── Dockerfile.monitor    # Contenedor del Monitor
│   └── Dockerfile.engine     # Contenedor del Engine
├── WM_FO/
│   ├── WM_FO.py              # Aplicación CLI del operario de campo
│   ├── activaciones/         # Ficheros JSON de planes de riego (FO-01..15)
│   └── Dockerfile            # Contenedor del Operario
├── scripts/
│   ├── run_multi_ws_windows.ps1   # Script PowerShell para 20 estaciones
│   ├── run_multi_fo_windows.ps1   # Script PowerShell para 15 operarios
│   ├── run_central_windows.bat    # Lanzador local Central
│   ├── run_ws_windows.bat         # Lanzador local 1 estación
│   └── run_fo_windows.bat         # Lanzador local 1 operario
├── docker-compose.yml        # Orquestación local nube (Kafka + Central)
└── docker-compose.lab.yml    # Orquestación de laboratorio (Estaciones + FO)
```

3.1. `utils/protocol.py`

- **Función:** Define la especificación binaria de comunicación por sockets TCP.
- `calcular_lrc(datos: bytes) -> bytes` **:** Recorre byte a byte el flujo aplicando `lrc ^= byte`. Devuelve un objeto `bytes` unitario.
- `empaquetar_trama(mensaje: str) -> bytes` **:** Codifica el texto a UTF-8, calcula el LRC sobre el payload y empaqueta `STX + payload + ETX + lrc`.
- `desempaquetar_trama(trama: bytes) -> (bool, str)` **:** Comprueba longitud mínima (3 bytes), valida que el primer byte sea `STX`, localiza el delimitador `ETX`, extrae el byte de LRC recibido y lo compara con el LRC recalculado. Si coinciden, devuelve `(True, mensaje_decodificado)`.
- `recv_exact_frame(sock, timeout=None) -> bytes` **:** Evita lecturas incompletas o mezclas de tramas. Primero extrae exactamente 1 byte con `sock.recv(1)`. Si es un byte de control aislado (`ENQ`, `ACK`, `NACK`, `EOT`), retorna inmediatamente. Si es `STX`, ejecuta un bucle acumulador hasta encontrar `ETX` y realiza una lectura final de 1 byte para incorporar el checksum `LRC`.

3.2. `utils/kafka_utils.py`

- **Función:** Abstrae la conexión a Apache Kafka y gestiona contingencias de arranque en frío.
- `crear_productor(broker: str, reintentos=10, espera=3) -> KafkaProducer` **:** Configura el serializador de valor a JSON UTF-8 y el serializador de clave a bytes UTF-8. Implementa un bucle de 10 reintentos ante la excepción `NoBrokersAvailable` (frecuente en despliegues con Docker o Railway donde Kafka tarda hasta 30 segundos en inicializar KRaft).
- `crear_consumidor(topico, broker, group_id, desde_inicio, ...)` **:** Crea el cliente consumidor suscriptor. Soporta múltiples tópicos simultáneos, asigna la política de offset (`earliest` o `latest`) y fija un parámetro crítico: `consumer_timeout_ms=1000`. Esto garantiza que los bucles `for msg in consumidor:` no queden bloqueados eternamente y puedan evaluar condiciones de salida o pausas periódicas.

3.3. `utils/db.py`

- **Función:** Capa de abstracción de datos para WM_Central.
- `Database.__init__(ruta_bd)` **:** Invoca `os.makedirs` sobre el directorio contenedor (evita fallos de "unable to open database file"), conecta con `check_same_thread=False`, establece `row_factory = sqlite3.Row` (permite acceso por nombre de columna) y ejecuta el DDL de creación de tablas (`estaciones`, `operarios`, `eventos`).
- `marcar_todas_desconectadas()` **:** Ejecutado en el arranque de Central para garantizar que ninguna estación figure artificialmente como activa tras un reinicio.
- `set_estado()`, `set_telemetria()`, `finalizar_riego()` **:** Métodos de actualización protegidos con `with _lock:`. `finalizar_riego` devuelve la estación a `DISPONIBLE` únicamente si su estado anterior era `REGANDO`, preservando situaciones de `FUGA` o `FUERA_DE_SERVICIO`.
- `log_evento(mensaje)` **:** Inserta el evento con timestamp en la base de datos y emite simultáneamente por consola estándar con el prefijo `[EVENTO]`.

3.4. `WM_Central/WM_Central.py` y `dashboard.py`

- **Función:** Núcleo de control de todo el sistema.
- **Hilos Concurrentes de WM_Central:**
- `servidor_sockets`: Socket TCP en `0.0.0.0:5000` con `SO_REUSEADDR`. Por cada conexión entrante de un Monitor lanza un hilo `atender_monitor`.
- `atender_monitor`: Implementa el diálogo de sockets. Procesa `ENQ` respondiendo `ACK`, procesa la trama `REGISTRO#id_ws#ubicacion`, da de alta la estación en BD como `DESCONECTADA` y almacena el socket en el mapa `conexiones_ws`. Si el socket se cierra o pasa 15 s sin latidos, marca la estación como `DESCONECTADA`.
- `hilo_peticiones`: Consume `riego-peticiones`. Valida existencia y disponibilidad de la estación. Si es apta, reserva el riego en `riegos_activos`, actualiza estado a `REGANDO`, emite `INICIAR_RIEGO` hacia el Engine y responde `AUTORIZADO` al operario. Si está ocupada o averiada, responde `DENEGADO` con el motivo exacto.
- `hilo_telemetria`: Consume `telemetria-riego`. Actualiza mediciones continuas en la BD. Al recibir `RIEGO_FINALIZADO`, actualiza la BD y emite el mensaje final `RESUMEN` al operario que solicitó el riego.
- `hilo_estado_ws`: Consume incidencias del Monitor. Ante un evento `FUGA`, pasa la estación a estado `FUGA`, interrumpe cualquier riego activo enviando `DETENER_RIEGO` al actuador y avisa al operario con un mensaje de evento `INTERRUMPIDO`.
- `hilo_broadcast`: Emite cada 2 segundos a `central-broadcast` la instantánea de todas las estaciones.
- `dashboard.py` **(Flask):** Servidor HTTP en puerto 8080. Mediante inyección de dependencias (`crear_app(db, fn_iniciar, fn_bloquear, fn_activar)`), el panel web queda desacoplado de Kafka. Expone `/api/state` para el refresco del frontend y `/api/action` para la ejecución de órdenes administrativas (`Regar`, `Bloquear`, `Activar`).

3.5. `WM_WS/WM_WS_M.py` (Monitor)

- **Función:** Supervisor de salud de la estación de riego.
- **Flujo de Conexión:**
- Conecta como cliente TCP a Central y ejecuta el registro con reintentos cada 5 segundos.
- Lanza el hilo `mantener_enlace_central` que emite un `ENQ` cada 5 segundos para mantener activo el canal. Si Central se cae, detecta el fallo y reintenta el registro indefinidamente hasta que vuelva.

- Abre un servidor TCP local en `ENGINE_PORT` (por ejemplo, 6001). Cuando el Engine se conecta, le transmite inmediatamente su identidad: `IDENTIDAD#id_ws#ubicacion`.
- Ejecuta un bucle cada 1 segundo enviando una trama `PING` al Engine. Si el Engine responde `KO` (fuga simulada) o no responde en 3 segundos (`timeout`), el Monitor publica en Kafka (`estado-ws`) el evento `FUGA`. Cuando el Engine vuelve a responder `OK`, publica `FUGA_RESUELTA`.

3.6. `WM_WS/WM_WS_E.py` (Engine)

- **Función:** Actuador de riego, generador de telemetría y gestor de fallos.
- **Mecanismos Internos:**
- `hilo_monitor`: Conecta al socket del Monitor, absorbe la identidad de la estación y responde a los *pings* con `OK` o `KO` según el flag booleano protegido `en_fuga`.
- `hilo_comandos`: Filtra del topic `comandos-estacion` las órdenes destinadas a su `id_ws` o a `"ALL"`. Si recibe `INICIAR_RIEGO`, lanza un hilo temporal `hilo_riego`.
- `hilo_teclado`: Permite al evaluador pulsar `k + Enter` para activar una fuga artificial (respondiendo `KO`) y `o + Enter` para restablecer el servicio normal.
- `hilo_auto_riego`: Controlado por las variables `AUTO_RIEGO_INTERVALO` y `AUTO_RIEGO_DURACION`. Si el intervalo es superior a 0, la estación lanza periódicamente riegos autónomos sin requerir intervención de operarios ni órdenes de Central.
- `hilo_riego`: Simula la apertura de la electroválvula. En un bucle de 1 segundo genera un caudal aleatorio realista entre 10 y 25 L/min, acumulando el volumen entregado:
  $$\Delta V = \frac{\text{caudal}}{60} \text{ litros/segundo}$$
  Emite periódicamente la telemetría a Kafka. Al finalizar (por tiempo, orden manual de parada, bloqueo administrativo o fuga), cierra la válvula y emite la trama `RIEGO_FINALIZADO` con el volumen acumulado final.

3.7. `WM_FO/WM_FO.py` (Field Operator)

- **Función:** Interfaz CLI del operario de campo.
- **Mecanismos Clave:**
- Comunicación exclusiva vía Kafka (sin sockets directos a Central).
- Generación de `request_id` (UUIDv4) para correlacionar respuestas de Central de forma no bloqueante.
- **Modo interactivo (Menú):** Permite listar estaciones (`1`), activar un riego manual (`2`) o cargar un plan por lotes (`3`).
- **Modo automático (`AUTO_MODE=1` o argumento de fichero):** Si se suministra la ruta a un fichero JSON en `sys.argv[3]`, el programa prescinde del menú interactivo, procesa secuencialmente todas las activaciones programadas esperando la finalización de cada riego y aplicando una pausa obligatoria de 4 segundos entre órdenes (según especificación de la práctica).

---

4. Guía de Despliegue y Manual de Evaluación para el Profesor

Esta sección proporciona las instrucciones exactas y comandos comprobados para que el profesor pueda desplegar y evaluar el proyecto en cualquiera de los entornos posibles.

Prerrequisitos de Software

- **Python:** Versión 3.10, 3.11, 3.12 o 3.13 instalada.
- **Docker Desktop:** Instalado y en ejecución (para Kafka y Central en local).
- **Librerías Python:** Definidas en `requirements.txt`:
  `txt kafka-python-ng>=2.2.2 Flask>=3.0.0`
  (Nota: Se utiliza `kafka-python-ng` para garantizar compatibilidad total con Python 3.12 y 3.13 sin errores de importación).

ESCENARIO 1: Despliegue 100% Local (Prueba Rápida en 1 Máquina)

Es la forma más rápida de verificar el proyecto sin depender de redes externas ni plataformas Cloud.

Paso 1.1: Clonar y preparar entorno Python

Abrir una terminal en la raíz del proyecto (`WaterManagement-SD`):

```
# En Windows:
scripts\setup_windows.bat

# En Linux / macOS:
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Paso 1.2: Arrancar la infraestructura Cloud local (Kafka + Central)

En la misma raíz del proyecto:

```
docker compose up --build
```

Esperar unos 20 segundos hasta ver en consola:

```
wm_central  | [WM_CENTRAL] Servidor de sockets escuchando en el puerto 5000...
wm_central  | [KAFKA] Escuchando estado de WS en 'estado-ws'...
```

Abrir el navegador web en: `http://localhost:8080`. El panel de monitorización cargará inicialmente vacío.

Paso 1.3: Lanzar una Estación de Riego (WS-01)

Abrir una nueva terminal con el entorno virtual activado:

```
# En Windows (script directo de 1 línea que abre Monitor y Engine):
scripts\run_ws_windows.bat 127.0.0.1 5000 WS-01 "River Park" 6001 127.0.0.1:9093

# En Linux / macOS (Terminal 1 - Monitor):
python WM_WS/WM_WS_M.py 127.0.0.1 5000 WS-01 "River Park" 6001 127.0.0.1:9093

# En Linux / macOS (Terminal 2 - Engine):
python WM_WS/WM_WS_E.py 127.0.0.1 6001 127.0.0.1:9093
```

Comprobación en el Panel Web: La estación `WS-01` pasará de color gris (`DESCONECTADA`) a verde (`DISPONIBLE`).

Paso 1.4: Lanzar el Operario de Campo (FO-01)

En otra terminal con el entorno virtual activado:

```
# En Windows:
scripts\run_fo_windows.bat 127.0.0.1:9093 FO-01

# En Linux / macOS:
python WM_FO/WM_FO.py 127.0.0.1:9093 FO-01
```

Aparecerá el menú interactivo. Pulsando la opción `1`, el operario listará las estaciones activas reportadas por el broadcast. Pulsando la opción `2`, solicitará regar `WS-01` durante 20 segundos.

ESCENARIO 2: Despliegue en 3 Ordenadores en Red Local (LAN)

Distribución física de la práctica en el aula sin depender de Railway:
- **PC-A (Nube Local):** Ejecuta Kafka + Central con `docker compose`.
- **PC-B (Estaciones):** Ejecuta los Monitores y Engines.
- **PC-C (Operarios):** Ejecuta los clientes `WM_FO`.

Paso 2.1: En PC-A (Nube Local)

1. Obtener la IP local de PC-A mediante `ipconfig` (Windows) o `hostname -I` (Linux). Supongamos `192.168.1.50`.
2. Habilitar los puertos en el cortafuegos de Windows (PowerShell Administrador):
   `powershell New-NetFirewallRule -DisplayName "WaterManagement" -Direction Inbound -Protocol TCP -LocalPort 5000,8080,9093 -Action Allow`
3. Crear un fichero `.env` en la raíz del proyecto en PC-A con:
   `env KAFKA_EXTERNAL_HOST=192.168.1.50`
4. Arrancar: `docker compose up --build`.
5. Verificar desde cualquier navegador de la red abriendo: `http://192.168.1.50:8080`.

Paso 2.2: En PC-B (Estaciones)

1. Crear el archivo `.env.lab` copiando `.env.lab.example`:
   `env CENTRAL_HOST=192.168.1.50 CENTRAL_PORT=5000 KAFKA_BROKER=192.168.1.50:9093`
2. Lanzar los contenedores de las estaciones con Docker:
   ```bash
   # Arrancar monitores en segundo plano:
   docker compose -f docker-compose.lab.yml --env-file .env.lab up -d ws01_monitor ws02_monitor

# Arrancar engines interactivos:
docker compose -f docker-compose.lab.yml --env-file .env.lab run --rm ws01_engine
   ```

docker compose -f docker-compose.lab.yml --env-file .env.lab run --rm ws02_engine
```

#### Paso 2.3: En PC-C (Operarios)

1. Configurar `.env.lab` con el mismo `KAFKA_BROKER=192.168.1.50:9093`.
2. Lanzar el operario interactivo:
   `bash docker compose -f docker-compose.lab.yml --env-file .env.lab run --rm fo1`

---

### ESCENARIO 3: Despliegue Oficial en Railway Cloud + Laboratorio

Despliegue de producción con la nube en Railway y estaciones/operarios en los puestos del laboratorio.

#### Paso 3.1: Configuración del Servicio `kafka` en Railway

1. Crear un proyecto vacío en Railway.
2. Añadir servicio: **New** → **Docker Image** con la imagen:
   `bitnamilegacy/kafka:4.0.0-debian-12-r10`
3. Renombrar el servicio exactamente a `kafka`.
4. En **Settings** → **Networking**, habilitar un **TCP Proxy** asignado al puerto `9093`. Railway generará una dirección tipo `algo.proxy.rlwy.net:12345`.
5. Añadir un volumen persistente montado en `/bitnami/kafka`.
6. En **Variables** → **Raw Editor**, introducir la configuración KRaft optimizada:
   ```
   env RAILWAY_RUN_UID=0 ALLOW_PLAINTEXT_LISTENER=yes KAFKA_CFG_NODE_ID=1 KAFKA_CFG_PROCESS_ROLES=controller,broker
   KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@127.0.0.1:9094 KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
   KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,EXTERNAL://:9093,CONTROLLER://:9094
   KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://${{RAILWAY_PRIVATE_DOMAIN}}:9092,EXTERNAL://${{RAILWAY_TCP_PROXY_DOMAIN}}:${{RAILWAY_TCP_PROXY_PORT}}
   KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT
   KAFKA_CFG_INTER_BROKER_LISTENER_NAME=PLAINTEXT KAFKA_CFG_OFFSETS_TOPIC_REPLICATION_FACTOR=1
   KAFKA_CFG_TRANSACTION_STATE_LOG_REPLICATION_FACTOR=1 KAFKA_CFG_TRANSACTION_STATE_LOG_MIN_ISR=1 KAFKA_CFG_LOG_RETENTION_MINUTES=60
   KAFKA_CFG_LOG_ROLL_MS=600000 KAFKA_CFG_LOG_RETENTION_CHECK_INTERVAL_MS=60000
   ```

#### Paso 3.2: Configuración del Servicio `wm_central` en Railway

1. Añadir servicio: **New** → **GitHub Repo** seleccionando el repositorio del proyecto.
2. Renombrar el servicio a `wm_central`. Dejar el campo *Root Directory* vacío.
3. En **Settings** → **Networking**:
4. Generar dominio público HTTP (puerto `8080`).
5. Crear un **TCP Proxy** en el puerto `5000` (para sockets de los Monitores). Generará algo como `proxy.rlwy.net:31147`.
6. Añadir un volumen persistente montado en `/app/data` (persistencia de `wm_central.db`).
7. En **Variables** → **Raw Editor**:
   ```
   env RAILWAY_DOCKERFILE_PATH=WM_Central/Dockerfile KAFKA_BROKER=${{kafka.RAILWAY_PRIVATE_DOMAIN}}:9092 SOCKET_PORT=5000 PORT=8080
   DB_PATH=/app/data/wm_central.db
   ```

#### Paso 3.3: Conectar los PCs del Laboratorio a Railway

Anotar los 3 parámetros de conexión de Railway:
- `CENTRAL_HOST`: Dominio del proxy de Central (ej. `proxy.rlwy.net`).
- `CENTRAL_PORT`: Puerto del proxy de Central (ej. `31147`).
- `KAFKA_BROKER`: Dominio y puerto del proxy de Kafka (ej. `algo.proxy.rlwy.net:12345`).

Rellenar el archivo `.env.lab` en los equipos del laboratorio con estos 3 valores y ejecutar los comandos de Docker o scripts batch tal como se describe en el Escenario 2.

---

### PRUEBA DE EVALUACIÓN MULTI-AGENTE: 20 Estaciones y 15 Operarios

Para comprobar la robustez y escalabilidad masiva del sistema ante el profesor:

1. **Lanzar 20 Estaciones Concurrentes (WS-01 a WS-20):**
   `powershell .\scripts\run_multi_ws_windows.ps1 -IpCentral 127.0.0.1 -PuertoCentral 5000 -KafkaBroker 127.0.0.1:9093`
   *El script abre de forma escalonada 40 terminales identificadas (20 Monitores + 20 Engines).*
2. **Lanzar 15 Operarios en Modo Automático (FO-01 a FO-15):**
   `powershell .\scripts\run_multi_fo_windows.ps1 -KafkaBroker 127.0.0.1:9093`
   *Cada operario ejecuta desatendidamente su plan de riego JSON en paralelo.*
3. **Observar en el Panel Web (`http://localhost:8080`):**
4. El refresco en tiempo real de caudales, volúmenes y operarios concurrentes.
5. La gestión de solicitudes denegadas por colisión cuando dos operarios intentan activar la misma estación.
   ```

6. El log cronológico de eventos en SQLite.

Protocolo de Comprobación de Casos Límite durante la Evaluación

Prueba a Realizar	Acción del Evaluador	Comportamiento Esperado del Sistema
Simulación de Fuga	En la terminal de un Engine (ej. `WS-01`), pulsar `k + Enter`.	La estación pasa inmediatamente a color **ROJO (** `FUGA` **)** en el panel web. Si estaba regando, Central emite `DETENER_RIEGO` al Engine y notifica `INTERRUMPIDO` al operario. La estación no acepta nuevas solicitudes de riego.
Resolución de Fuga	En la misma terminal del Engine, pulsar `o + Enter`.	El Monitor detecta la respuesta `OK`, notifica `FUGA_RESUELTA` a Central y la estación retorna a color **VERDE (** `DISPONIBLE` **)**.
Bloqueo Administrativo	Pulsar el botón **Bloquear** en el panel web sobre una estación.	Pasa a color **NARANJA (** `FUERA_DE_SERVICIO` **)**. Cualquier intento de un operario de regar en ella es respondido con `DENEGADO`.
Desconexión Física	Cerrar la ventana del Monitor de una estación.	En ≤ 15 segundos salta el timeout en Central y la estación pasa a color **GRIS (** `DESCONECTADA` **)**. Los botones de control quedan deshabilitados con advertencia. Al relanzar el Monitor, la estación se recupera sola.
Reinicio de Central	Reiniciar el proceso `WM_Central` o forzar *Redeploy* en Railway.	Al arrancar, Central ejecuta `marcar_todas_desconectadas()` pasando todas las filas a gris. En 5–15 segundos, los Monitores activos re-ejecutan el handshake socket y la Central recupera el estado real exacto de toda la red sin inconsistencias.
Llegada Tardía de Operario	Lanzar un operario cuando su riego ya ha terminado.	Gracias a `desde_inicio=True` en Kafka, el operario lee el topic histórico filtrando por su `id_operador` y muestra el resumen volumétrico final.

---

5. Conclusiones

La solución implementada demuestra un dominio integral de los principios de Sistemas Distribuidos:
- **Desacoplamiento Efectivo:** Separación estricta entre capa de transporte orientada a conexión (sockets) y bus de eventos persistente (Kafka).
- **Consistencia y Atomicidad:** Gestión segura de la concurrencia multihilo con cerrojos atómicos sobre SQLite y estados protegidos con eventos de sincronización.
- **Robustez Operativa:** El sistema tolera caídas y reinicios en cualquiera de sus capas (actuador, supervisor, central o clientes) recuperando la consistencia de forma completamente autónoma.
- **Facilidad de Demostración:** Automatización completa mediante scripts PowerShell, contenedores Docker parametrizados y panel web en tiempo real.